
How to Prepare a PRD for Vibe Coding

Week 3 — Extra Material

AI cannot infer from omission. What you don't specify, AI will invent.

Instructor: Goker Ezberci

Vibe Coding 101: From Idea to Shipped Product

github.com/goker/vibe-coding-101-for-software-engineers



Why PRDs Matter in Vibe Coding

The difference between building what you want and building what AI guesses

✗ Without a PRD

"Build me a todo app"

AI adds authentication you didn't want

AI picks PostgreSQL (you wanted JSON)

AI builds React frontend (you wanted CLI)

3 hours wasted, start over

✓ With a PRD

7 sections tell AI exactly what to build

Non-Goals prevent scope creep

Technical constraints lock the stack

Success criteria = testable "done"

Ship in one focused session

The 7 Sections of a PRD

Every PRD follows this exact structure. No more, no less.

1 Overview

What, Who, Why

2 Core Features

3–5 specific MVP features

3 Non-Goals

What you WON'T build

4 Tech Constraints

Versions, stack, storage

5 Success Criteria

Testable checkboxes

6 Phases

Step-by-step with verification

7 Agent Rules

Always / Ask First / Never

Section 1: Overview

Define What, Who, and Why in one sentence each

Overview

What	A CLI todo app for managing tasks	
Who	Developers who prefer terminal	
Why	Quick task mgmt without leaving CLI	



Common Mistake

- ❑ “An app for users” — vague, no target audience
- ❑ “A CLI todo app for developers who prefer terminal workflows”



Checklist

- ❑ What: One sentence describing the project
- ❑ Who: Specific target user (not “users”)
- ❑ Why: Clear problem being solved

Section 2: Core Features (MVP)

3–5 specific features with command examples

Core Features

1. **Add Task:**
todo add "Buy groceries" → Creates with auto ID
2. **List Tasks:**
todo list → Shows all tasks with [] or [x]
3. **Complete Task:**
todo done 3 → Marks task #3 as complete

- ❑ “Make it work well” — vague, unmeasurable
- ❑ todo list shows all tasks with [] or [x] prefix

1
2
3

Checklist

- ❑ 3–5 features maximum
- ❑ Each feature is numbered
- ❑ Each has a command/usage example
- ❑ Features are specific (not vague)

Section 3: Non-Goals (CRITICAL!)

The most important section. AI reads your PRD and wonders: "Should I add authentication?"

Will NOT Build

- ✗ User authentication or login
- ✗ Due dates or reminders
- ✗ Cloud sync or backup
- ✗ Mobile app or web interface
- ✗ Multi-user collaboration

Will NOT Use

- ✗ Database (SQLite, PostgreSQL, etc.)
- ✗ External APIs
- ✗ Rich terminal UI libraries
- ✗ Docker or containerization
- ✗ Any paid services



Rule: If you wouldn't build it in v1, list it as a Non-Goal. When in doubt, add it.

Section 4: Technical Constraints

Lock down the stack — exact versions, no ambiguity

Technical Constraints

Language	Python 3.11+	
Framework	None (stdlib only)	
Dependencies	None	
Data Storage	JSON at ~/.todo/tasks.json	
Testing	pytest, minimum 3 tests	

☐ “Any framework is fine” — AI will pick the heaviest one

☐ “Python 3.11+, stdlib only, no external dependencies”



Checklist

- ☐ Language with exact version
- ☐ Framework specified (or None)
- ☐ Dependencies listed (or None)
- ☐ Data storage location
- ☐ Testing framework specified

Section 5: Success Criteria

Testable checkboxes for “done” — no vague language allowed

Success Criteria

- [] todo add "text" creates task with unique ID
- [] todo list shows all tasks with status
- [] Data persists between sessions
- [] All tests pass

- ☐ “Works well”
- ☐ “Is fast”
- ☐ “Handles edge cases”

Every criterion must be a yes/no test you can run from the terminal.

Map each criterion to a Core Feature from Section 2.



Checklist

- ☐ Each criterion is testable
- ☐ No vague terms
- ☐ Includes specific commands
- ☐ Maps to Core Features

Section 6: Implementation Phases

Step-by-step build plan with verification at every step

Phase 1: Core Data Model

Goal: Set up project + implement task storage

Tasks:

1. Create project structure
2. Define Task dataclass
3. Implement JSON read/write

Verification:

```
python todo.py add "Test"  
cat ~/.todo/tasks.json # Shows task
```

Deliverables: todo.py, JSON utilities



Each Phase Has

Goal

One sentence

Tasks

Numbered list

Verification

Copy-paste commands

Deliverables

Files created



Missing verification is the #1 mistake. Every phase needs a command you can copy-paste to confirm it works.

Section 7: Agent Rules

Decision-making guardrails for AI — Always, Ask First, Never

Always

Do this automatically

- ☐ Validate task ID before ops
- ☐ Handle file I/O errors
- ☐ Show help text on bad input
- ☐ Use type hints everywhere

Ask First

Get human approval

- ☐ ⚠ Before changing data schema
- ☐ ⚠ Before adding new commands
- ☐ ⚠ Before any refactoring
- ☐ ⚠ Before changing file paths

Never

Hard stop, don't do this

- ☐ Use external packages
- ☐ Add features not in PRD
- ☐ Skip writing tests
- ☐ Delete user data silently

3 Mistakes That Waste Hours

Every one of these leads to AI building the wrong thing

1

Vague Features

- ❑ "Make it work well"
- ❑ todo list shows all tasks with [] or [x]

Include exact commands with expected output

2

Missing Non-Goals

- ❑ (No Non-Goals section)
- ❑ "Will NOT build: auth, database, cloud sync"

AI fills gaps with guesses — specify what's out

3

No Verification

- ❑ "Phase 1: Set up project"
- ❑ Verification: `python todo.py add "Test"` → Shows ID

Every phase needs a copy-paste command to test

PRD Quality Checklist

Check every box before you start coding. If ANY is missing, fix it first.

Overview

- ☐ What / Who / Why (all three)

Features

- ☐ Numbered (3–5 max)
- ☐ Each has command example

Non-Goals

- ☐ 5+ things you WON'T build
- ☐ Technologies you WON'T use

Constraints



- ☐ Exact versions

- ☐ Framework or None

Final Check: Re-read entire PRD. Ask: “What’s missing that AI might assume?”

Success

- ☐ Testable criteria
- ☐ Maps to features

Phases

- ☐ 2–4 phases
- ☐ Verification commands

Agent Rules

- ☐ Always (3+) / Ask First (2+) / Never (3+)

24 Example PRDs

Pick one that matches your skill level and build it this week

Beginner (2–3 hours)

1. Todo CLI
2. Pomodoro Timer
3. Password Generator
4. Flashcard Quiz
5. Unit Converter
6. Daily Journal
7. Markdown Converter
8. Expense Tracker

Intermediate (4–8 hours)

1. Recipe API
2. Bookmark Manager
3. Habit Tracker
4. File Organizer
5. Screenshot Tool
6. RSS Aggregator
7. Commit Message Skill
8. Workout Tracker

Advanced (1–4 weeks)

1. AI Code Review
2. Real-time Chat
3. Finance Dashboard
4. Task API (Teams)
5. SaaS Starter
6. Presentation Poll

All 24 examples available at: github.com/goker/vibe-coding-101-for-software-engineers/.../templates/examples

Key Takeaways

- 1  AI cannot infer from omission — be explicit about everything
- 2  Non-Goals are critical — list what you WON'T build
- 3  Use Agent Rules — define Always / Ask First / Never boundaries
- 4  Include verification — every phase needs test commands
- 5  Be specific — commands with expected output, not vague descriptions